Technical report — ParlaSchema

- **Track:** Track 2A — OpenParlData
- **Event:** Online
- **Team:** Not provided
- **Demo:** Synthetic offline CLI replay via `cd track_2a && make run`; video/deployment link not available.

1. Summary

This checkout delivers a Python/Pydantic schema and provenance foundation plus a scoped Apertus block-classification precursor. It validates source-bound records, preserves original text and evidence references, exports a versioned JSON Schema, and provides a strict offline replay harness. It does **not** deliver the requested PDF-to-schema converter, automatic document segmentation, document profiles, or a verified eight-field extraction pipeline. The accompanying feasibility evidence is intentionally partial and negative: seven diagnostic series used 78 historical calls, but real labels were provisional, drawn from four known development PDFs, with no human-approved gold or held-out score. This report therefore does not claim track completion, benchmark accuracy, or general Apertus capability.

2. Architecture

The implementation uses Python 3.12, `uv`, Pydantic 2 and pytest. The main components are:

- `schema.py`: document-scoped eight-field models, typed status values, source evidence, local IDs and explicit document outcomes.
- `provenance.py`: exact supplied-page/quote checks. It does not fetch, normalize, OCR, infer semantics or prove PDF completeness.
- `reference.py`: machine-draft and human-review structures. No verified real goldset is included.
- `schema_export.py`: deterministic `schemas/extraction-v0.1.json` export.
- `block_classification.py`: immutable source document/block inputs, exact ranges, context binding, versioned prompts and strict category parsing.
- `block_evaluation.py`: offline replay with fixed denominators and separate correct/incorrect/abstention/structure/transport outcomes.
- `block_live.py` and `block_http.py`: explicit opt-in exploratory transport tooling. They are not used by the default demo and do not implement PDF conversion.

The actual domain pipeline is intentionally absent. The repository does not segment PDFs, recover tables/charts, create document profiles, select blocks, populate the eight fields, or evaluate semantic extraction against approved references.

3. Use of Apertus

The code contains an explicit Apertus-compatible HTTP transport and versioned category-only prompts in `block_classification.py`: `category-json-v0.1` (default), `category-word-v0.1` (explicit opt-in) and `source-function-v0.2` (opt-in input/semantic method using the word contract). Historical private diagnostics used `swiss-ai/Apertus-v1.5-70B` on CSCS through a hosted OpenAI-compatible endpoint, for inference only, without training or a second model. Settings were temperature 0, top_p 1, 64 output tokens, 60-second socket timeout and a 1,200-second scheduling budget; no retries, repair or fallback. These are historical settings, not current provider capability verification. Those runs are not required to execute this checkout and are not reproduced by `make run`. Credentials, private source text, raw responses and approval records are outside the repository.

The default path is synthetic offline replay. It makes no model request and does not present scripted responses as live extraction. The opt-in `prepare-block-test` path is dry-run by default and has explicit network, plan, source-transmission and budget guards.

4. Data

Public repository data is limited to versioned source metadata, inspection metadata and wholly synthetic fixtures. Real PDFs, extracted text, quotes, machine drafts, labels and review ledgers remain outside the repository. Manifest metadata records pending redistribution and parser suitability; a public URL or metadata license is not treated as permission to redistribute third-party PDF/text content or transmit it to a model.

Code is Apache-2.0 under the repository `LICENSE`. Documentation is CC BY 4.0. OpenParlData.ch is identified as CC BY 4.0 for the relevant public source metadata; this does not relicense private or third-party source material.

5. Evaluation

The reproducible public evaluation is only the synthetic replay:

```
cd track_2a
uv run --frozen openparl-extractor evaluate-blocks \
  tests/fixtures/block-replay-synthetic.json
```

Its three scripted cases exercise one correct result, one abstention and one transport error. It measures harness accounting, not model quality.

Historical private diagnostics are reported as separate series:

Series	Calls	Reportable observation
Real blocks, JSON contract	18	18 structure errors under that contract
Synthetic role controls	4	4/4 valid JSON format
Synthetic JSON classification	4	1 valid format, 3 structure errors
Synthetic word classification	4	4/4 agreement on invented controls
Real blocks, word contract	18	11 correct, 7 incorrect against provisional agent labels; all outputs valid
Context/question-header matrix	18	3 correct, 15 incorrect on deliberately selected known error cases
Paired function-prompt comparison	12	Each arm: 4 correct, 2 incorrect on 6 cases; identical categories across all pairs

The 78 calls use different tasks and denominators; they are not combined into an accuracy score. The real word-contract series used 18 blocks from four known development PDFs. There is no approved goldset, no held-out score, no positive request/expected-unclear coverage and no eight-field score. Three limited human decisions do not establish an approved global category policy. The inputs were manually bounded; automatic segmentation was neither tested nor excluded as a possible cause in future pipelines. Quote occurrence and source hashes do not prove semantic correctness or completeness. Earlier A/B and whole-document anchor tests are separate feasibility evidence, not part of the 78-call table. Individual context variants produced isolated changes, but no robust improvement; the function-prompt pair comparison showed no observed arm difference in its small selected matrix. These observations are not a general claim about Apertus. The old-prompt arm used the legacy word prompt on the same newly derived payload as the new prompt, not unchanged historical legacy requests. It does not isolate a single prompt rule or prove the benefit of header transfer. No best-variant selection or retrospective rescoring is used. A rules-only semantic baseline was not implemented, so central Apertus added value remains unproven. Monetary costs are unknown; confirmed covered quota is not a claim of zero cost. The internal historical 80% gate is not official track acceptance.

6. Limitations

- No PDF/OCR/layout-to-block segmentation or PDF-to-schema converter.
- No profile implementation for document families, page continuation or repeated question/answer headers.
- No verified end-to-end extraction, evaluator or human-approved references.
- Charts, tables, outlined text and source-version completeness remain unresolved in private inspection.
- Historical live results are not independently reproducible from checkout because their private inputs and labels are excluded.
- Hybrid routing and A1 semantic contrast work stopped at offline hypotheses. Two additional development sources were privately inspected, not sent to Apertus; they do not establish non-lexical request contrasts or source rights.
- Team identity, demo video/deployment and final submission revision were not provided in repository data; these remain administrative gaps.

The negative/partial feasibility result is presented openly. It does not replace the missing converter deliverable.

7. Reproducibility

Default key-free demo:

```
cd track_2a
make run
```

This starts a Docker container with the public synthetic fixture and prints an `offline_replay` report labelled `synthetic`. The fixture includes a scripted transport error as an accounting case; it is not a current provider error. No API key or model network access is required for demo execution. An initial Docker build may download public image/dependency artifacts. Make targets ignore `.env` files; optional configuration validation forwards explicitly exported shell settings and remains separate:

```
make check-config
```

Local tests:

```
cd track_2a
uv run --frozen pytest -q
```

The block-classification implementation was introduced in `2ee7a5db0c74cf96c88d13262f967ce43fc32700`; the submission packaging and this report are later changes. Record `git rev-parse HEAD` for the revision you actually run rather than treating that historical commit as the demo revision.

8. Next steps

First, confirm the accepted submission format and provide team/demo metadata. Second, review the small Docker demo and report wording. Building a real converter would be a separate scope decision requiring source rights, segmentation rules, approved references and a new evaluation plan. No further model calls, profile work or private-source publication is implied here.

License

Code: Apache-2.0. Documentation: CC BY 4.0. OpenParlData source metadata: CC BY 4.0. Source-PDF/text redistribution remains subject to separate rights review; none of these licenses grants blanket source transmission permission.

References

- `README.md` — setup, current scope and offline demo.
- `docs/block-classification.md` — block contract and historical procedure boundaries.
- `docs/category-word-contract.md` — explicit output/method versions.
- `docs/schema.md` — schema/provenance semantics.
- `docs/goldset.md` — private-review and approval boundaries.
- `data/manifest.json` and `data/pdf-inspection.json` — public metadata only.